Design and Implementation of a Movie Recommendation and Tracker System

Coursework 3 Project Report

Group 24

Student Name	Student ID
Churui Yang	[2472765]
Yuhui Pang	[2471217]
Siqi Cai	[2469897]
Qingxue Lu	[2469580]
Mingyu Cheng	[2473726]

Semester 1, 2025-26

Contents

1 System Design and Object-Oriented Principles **3**
1.1 Introduction . . . 3
1.2 Encapsulation and Composition . . . 3
1.3 Class Structure . . . 3

2 Recommendation Algorithm Description **3**
2.1 Overview of Recommendation Strategy . . . 3
2.2 Handling the Cold Start Problem . . . 4
2.3 Strategy for Active Users . . . 4
2.3.1 Feature Extraction & Scoring Models . . . 5
2.3.2 Adaptive Weighting Mechanism . . . 5
2.3.3 Strategy Toggle Mechanism (User-Controlled Feature Selection) . . . 6

3 Application Features & UI Implementation **6**
3.1 Authentication, Session Management & Security . . . 6
3.1.1 Login, Registration, and Session State . . . 6
3.1.2 Password Storage Security: One-way Hashing . . . 7
3.1.3 Usability Enhancement: Password Visibility Toggle . . . 7
3.2 Watchlist & History Tracking . . . 8
3.3 Recommendation Visualization . . . 8

4 File Handling **9**
4.1 File Reading . . . 9
4.2 File Saving . . . 9

5 Exception Handling and System Robustness **9**
5.1 Data Persistence & File Corruption Recovery . . . 10
5.2 User Input Validation . . . 10
5.3 Algorithmic ”Cold Start” Handling . . . 10

6 Testing Strategy and Results **10**
6.1 Functional Testing . . . 10
6.2 Algorithm Verification . . . 12

7 Project Management & Ethical Assessment **12**
7.1 Software Engineering Team Practices . . . 12
7.2 Ethical Assessment & Privacy Considerations . . . 13

1 System Design and Object-Oriented Principles

1.1 Introduction

This report presents the design, implementation, and evaluation of our Movie Recommendation and Tracker System. The purpose of the system is to allow users to manage their personal movie watch lists, maintain a viewing history, browse a movie database, and receive personalized movie recommendations based on their past behaviour.

The project emphasizes object-oriented programming, file processing, exception handling, and software engineering teamwork. Our system additionally includes a graphical user interface, a multi-strategy recommendation engine, and functionalities for creating new user accounts and changing passwords, as well as encrypting them before saving to CSV files as advanced features.

1.2 Encapsulation and Composition

We strictly adhere to encapsulation principles throughout the system, which is one of the three core principles of OOP. Variables in classes such as `User` and `Movie` are declared private to prevent unauthorized external modification, with access controlled via public getter and setter methods.Moreover, the system demonstrates composition. For example, a `User` object contains a `Watchlist` and a `History`. Instead of placing all list management logic inside the `User` class, we delegated these responsibilities to dedicated `Watchlist` and `History` classes. This modular approach makes our code easier to maintain and extend.

1.3 Class Structure

Overall, the main classes include `Movie`, `User`, `Watchlist`, `History`, `RecommendationEngine`, `Movie FileHandler`, and `UserFileHandler`, along with the `MovieAppGUI` as the entry point for the graphical user interface.

In terms of class interactions, the `MovieAppGUI` class serves as the primary controller that coordinates the flow of data. Upon startup, it calls the `loadMovies` and `loadUsers` methods from the file handler classes to populate the system's memory. During the user session, the GUI interacts directly with `User` objects to modify their Watchlist and History states. When recommendations are requested, the GUI passes the current user and movie list to the `RecommendationEngine`, which computes scores and returns a ranked list. Eventually, file handlers are invoked again to save any changes when the application is closed.

2 Recommendation Algorithm Description

2.1 Overview of Recommendation Strategy

The recommendation system implements a hybrid dual-strategy engine designed to handle both new users (cold start) and active users (personalized needs). The system intelligently switches between a **Stochastic Top-Rated Strategy** for users with no interaction history and a **Dynamic**

Weighted Content-Based Filtering algorithm for users with established preferences. This ensures that the system remains robust and effective across different stages of the user lifecycle.

2.2 Handling the Cold Start Problem

When a new user registers or has cleared their watch history and watchlist, the system triggers the **Cold Start Protocol**. Instead of returning a static list of "Best Movies," the system employs a randomized pooling mechanism to ensure diversity.

The system retrieves all movies from the database and ranks them by their global rating in descending order. The top 20 highest-rated movies are selected to form the **Candidate Pool**. The choice of 20 movies strikes a balance between quality assurance and diversity introduction:

- If the candidate pool is too small, it would result in nearly identical recommendations for all new users, causing severe recommendation fatigue and eliminating the benefit of randomization.
- If the candidate pool is too large, it would dilute the quality threshold, potentially recommending mediocre movies to new users, which could harm first impressions and user retention.

Empirically, the top 20 movies maintain consistently high ratings while providing sufficient variability. To prevent recommendation fatigue, the pool is randomly shuffled using `Collections.shuffle()` before selection. This ensures that each user perceives a unique and fresh recommendation experience.The top N movies from the shuffled pool are presented to the user as recommendations. This design effectively eliminates the perception of "cookie-cutter" recommendations.

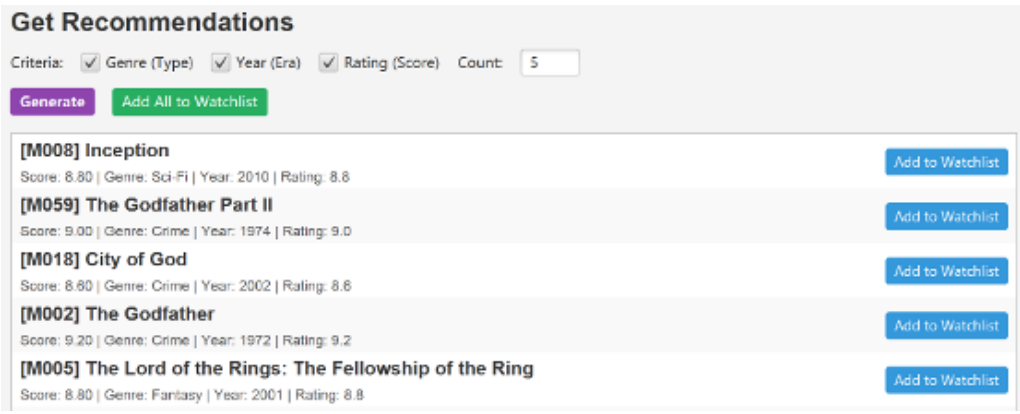


Figure 1: Get Recommendations Interface showing Cold Start results

2.3 Strategy for Active Users

For active users with interaction history, the system constructs a dynamic user profile. The recommendation score for a candidate movie m is calculated using a Linear Combination Model with adaptive weights.

The final recommendation score is determined by the weighted sum of three feature similarity sub-scores:

$$Score_m = w_{genre} \cdot S_{genre}(m) + w_{year} \cdot S_{year}(m) + w_{rating} \cdot S_{rating}(m) \tag{1}$$

where $S_k(m)$ is the scoring function for feature dimension k (Genre, Year, Rating) and W_k is the dynamic weight for dimension k, calculated based on the user's historical consistency ($\sum w_k = 1$).

2.3.1 Feature Extraction & Scoring Models

We quantify the match between a movie and the user's profile using three distinct statistical methods:

1. Genre Preference Score: Represents the frequency probability of a genre in the user's history. It promotes movies that align with the user's most-watched categories.

$$S_{genre}(m) = P(g_m) = \frac{\text{Count}(g_m)}{\text{Total Watched Movies}} \tag{2}$$

2. Year Preference Score: Assuming the user's preference for movie eras follows a Normal Distribution, we calculate the mean year (μ) and variance (σ^2) of their history. We use a Gaussian Decay Function to measure similarity. The closer a movie's year is to the user's average, the higher the score.

$$S_{year}(m) = e^{-\frac{(Year_m - \mu)^2}{2\sigma^2}} \tag{3}$$

3. Rating Quality Score is a normalized value of the movie's global rating, ensuring high-quality movies maintain a baseline advantage regardless of genre or year.

$$S_{rating}(m) = \frac{Rating_m}{10.0} \tag{4}$$

2.3.2 Adaptive Weighting Mechanism

The system does not use fixed weights. Instead, it employs an **Inverse Diversity Logic**: if a user's behavior is highly concentrated in a specific dimension, that dimension is assigned a higher weight. We calculate three diversity metrics (D):

- **Genre Diversity (D_{genre})**: Based on the Herfindahl-Hirschman Index (HHI).

$$D_{genre} = 1 - HHI = 1 - \sum_{i=1}^{n} p_i^2 \tag{5}$$

- **Temporal Diversity (D_{year})**: Normalized by a 20-year generation span.

$$D_{year} = \min\left(\frac{\sigma}{20}, 1.0\right) \tag{6}$$

- **Rating Diversity (D_{rating})**: Measuring the proportion of non-high-rated movies.

$$D_{rating} = 1 - \frac{\text{Count}(Rating \geq 7.0)}{\text{Total Movies}} \leftarrow \tag{7}$$

These diversity metrics are transformed into normalized weights using a Softmax-like exponential function with scaling hyperparameter $\lambda = 5.0$:

$$w_k = \frac{e^{(1-D_k)\cdot\lambda}}{e^{(1-D_{genre})\cdot\lambda} + e^{(1-D_{year})\cdot\lambda} + e^{(1-D_{rating})\cdot\lambda}} \tag{8}$$

This formulation guarantees sum w = 1 and ensures weights dynamically adapt to evolving user preferences.

2.3.3 Strategy Toggle Mechanism (User-Controlled Feature Selection)

The system grants users explicit control over recommendation dimensions (Genre, Year, Rating) via GUI checkboxes, enhancing transparency and enabling exploration beyond established habits. When a feature is deselected, the algorithm applies a **Conditional Zero-Forcing** strategy: the exponential term for the disabled dimension is set to zero, and the remaining weights are automatically renormalized. Crucially, the disabled weight is **proportionally redistributed** to active dimensions based on their inherent concentration intensities. This mechanism ensures mathematical consistency, preventing any single dimension from gaining artificial dominance simply due to the exclusion of another (e.g., disabling 'Year' smoothly shifts influence to 'Genre' and 'Rating').

3 Application Features & UI Implementation

This section details the user interface design and the implementation of core features using JavaFX. The application follows a scene-based architecture where the Stage switches between Login, Register, and the Main Application scenes.

3.1 Authentication, Session Management & Security

The application provides a complete authentication workflow, combining usability (clear input and feedback) with basic security controls for local CSV-based persistence.

3.1.1 Login, Registration, and Session State

- **Login & Session State:** Users authenticate via a `TextField` (username) and `PasswordField` (password). After a successful login, a global `currentUser` object is set, ensuring that all subsequent actions (e.g., watchlist/history updates) operate on the correct user and preventing cross-user data contamination.
- **Registration Logic:** The registration process validates username uniqueness against `users.csv` and includes a "Confirm Password" check to reduce account creation errors.
- **Password Update:** The system supports a "Change Password" flow with three-step validation (verify old password, validate new password, confirm new password).

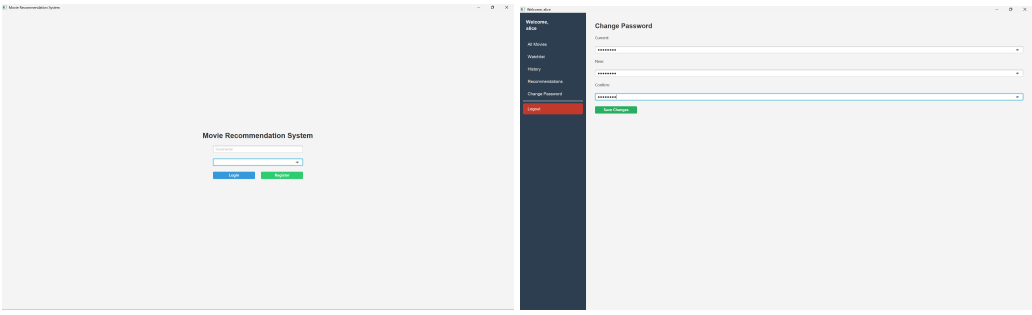


Figure 2: Login interface and change-password workflow

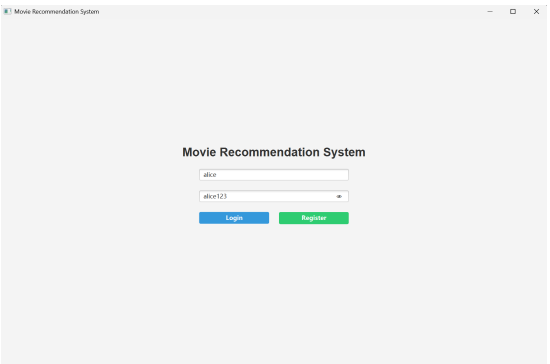
3.1.2 Password Storage Security: One-way Hashing

To enhance security, the system does not store user passwords in plain text in `users.csv`. Instead, **passwords are transformed into a one-way hashed string** before being written to disk. During authentication, the input password is hashed using the same procedure and compared against the stored hash value. This ensures that the original password cannot be recovered from the CSV file even if it is accessed.

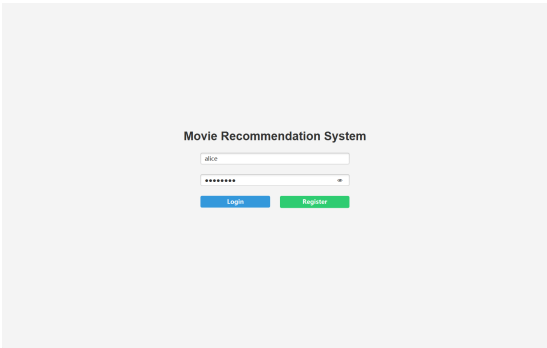
Implementation-wise, the hashing logic is encapsulated within the `User` class. Therefore, the file persistence layer does not need special-case code: `UserFileHandler` simply writes whatever is returned by `user.getPassword()`, and this value is guaranteed to be in hashed form when persisted.

3.1.3 Usability Enhancement: Password Visibility Toggle

In addition to default password masking, the login interface provides a **visibility toggle**. This allows users to temporarily switch between hidden and visible input, reducing typing mistakes for complex passwords while keeping the default state secure. Figure 3 illustrates the two states.



(a) Visible



(b) Hidden

Figure 3: Password visibility toggle for improved usability

3.2 Watchlist & History Tracking

The application provides two dedicated views: **My Watchlist** and **Watch History**.These views combine smart logic with batch operation capabilities.

- **Smart History Transition:** When a user marks a movie as ”Watched”, the system captures the date, adds the movie to History, and automatically removes it from the Watchlist to prevent redundancy.
- **Batch Operations:** A ”Clear All History” function uses a Red-styled button (#c0392b) to visually warn the user and checks if the history is empty before executing.

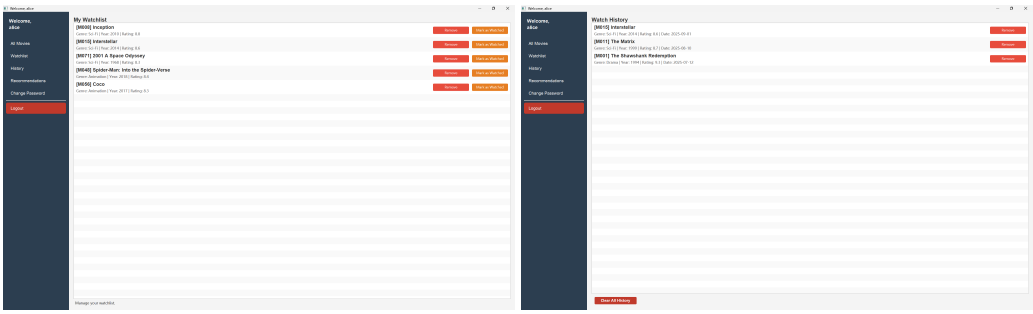


Figure 4: Watchlist and History View highlighting the ”Clear All” button

3.3 Recommendation Visualization

To enhance readability, the application uses **custom Cell Factories** to create rich graphical rows. Each movie entry displays the Title in Bold followed by detailed metadata. Users can also interact with recommendation controls to toggle specific weights.

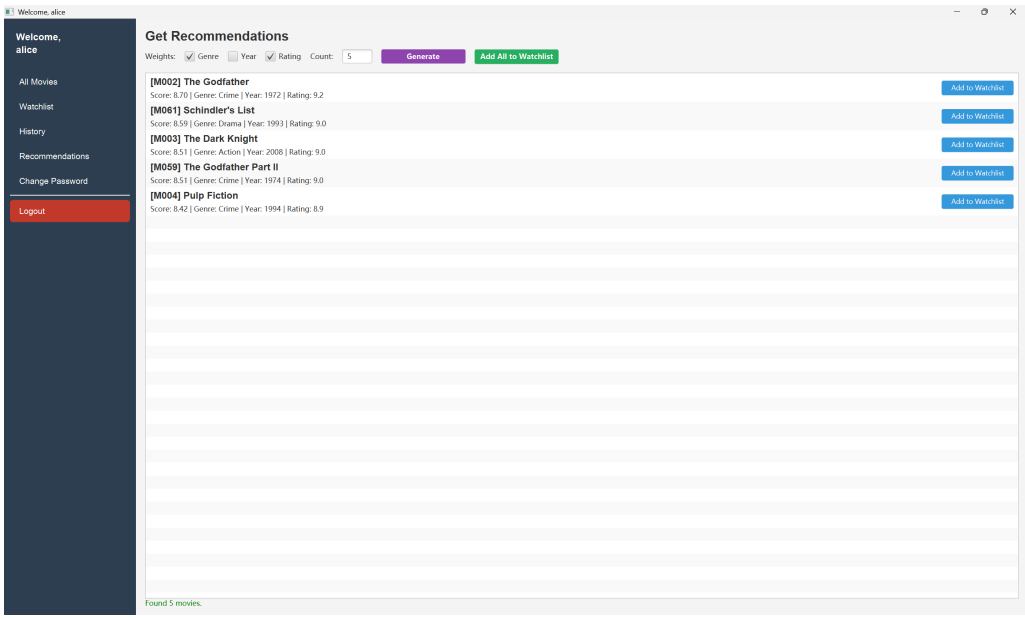


Figure 5: Recommendation UI with custom cell formatting

4 File Handling

The system **separates file input/output logic from the user interface** by using two dedicated utility classes: `MovieFileHandler` and `UserFileHandler`. This design keeps file processing independent from the GUI logic, improving code readability and maintainability. All persistent data is stored in CSV files, and Java's standard java.io library is used to load and save both movie data and user data.

4.1 File Reading

When loading data from CSV files, the system focuses on both correctness and robustness. Movie and user data are read using `BufferedReader`, which allows efficient line-by-line processing of large files. To ensure that file resources are always released correctly, file streams are closed in a finally block, even if an exception occurs during reading.
Since external files may contain imperfect data, basic sanitization is applied during parsing. Each field extracted from a CSV line is trimmed to remove unnecessary whitespace, preventing parsing errors when converting values such as year or rating. Before accessing fields, the program checks the length of the parsed array to avoid runtime errors caused by malformed or incomplete lines.
For user data, the system supports more complex structures. While commas are used as the primary delimiter between user fields, semicolons are used to separate movie titles inside watchlists and viewing histories. This approach allows lists to be stored in a standard CSV format while still being correctly reconstructed in memory.

4.2 File Saving

When saving user data back to disk, the system converts in-memory objects into a structured CSV representation.
To improve efficiency, `StringBuilder` is used when assembling each line of user data, avoiding repeated string concatenation inside loops. Because CSV files cannot directly store lists, a custom helper method converts watchlists and histories into semicolon-separated strings before writing them to the file.
To reduce unnecessary disk operations, file saving is **triggered only at important events**, such as user registration or when the application is closed. This ensures that user data is saved reliably without affecting the runtime performance of the application.

5 Exception Handling and System Robustness

To ensure a smooth user experience and prevent application crashes during runtime, the system implements a multi-layered exception handling strategy. This chapter details how the application manages unexpected situations across data persistence, user interaction, and algorithmic logic.

5.1 Data Persistence & File Corruption Recovery

We implemented a **line-level fault tolerance mechanism**. Instead of crashing on a corrupted line, the system catches specific exceptions inside the read loop.

```
catch (NumberFormatException e) {
    System.out.println("Warning: Invalid data format in line: " + line);
    // The loop continues to the next line automatically
}
```

Listing 1: Fault Tolerance in File Reading

5.2 User Input Validation

The GUI proactively validates inputs. For example, the Recommendation Count input expects an integer.

```
catch (NumberFormatException ex) {
    updateStatus(statusLabel, "Invalid number.", Color.RED);
}
```

Listing 2: Input Validation in GUI

5.3 Algorithmic "Cold Start" Handling

To prevent NaN propagation or ArithmeticException when calculating statistics on an empty history, the engine checks if the user data is empty and **falls back to top-rated movies**.

```
// If uniqueIds is empty, fallback to top-rated
if (uniqueIds.isEmpty()) {
    return getTopRatedMovies(user, n);
}
```

Listing 3: Cold Start Fallback

6 Testing Strategy and Results

6.1 Functional Testing

We conducted comprehensive **Black-Box testing** to verify that the system functionality aligns with the requirements. We tested the login function to ensure valid credentials grant access while invalid ones trigger an error message. The following images show the testing result (using bob's account with correct and wrong password).

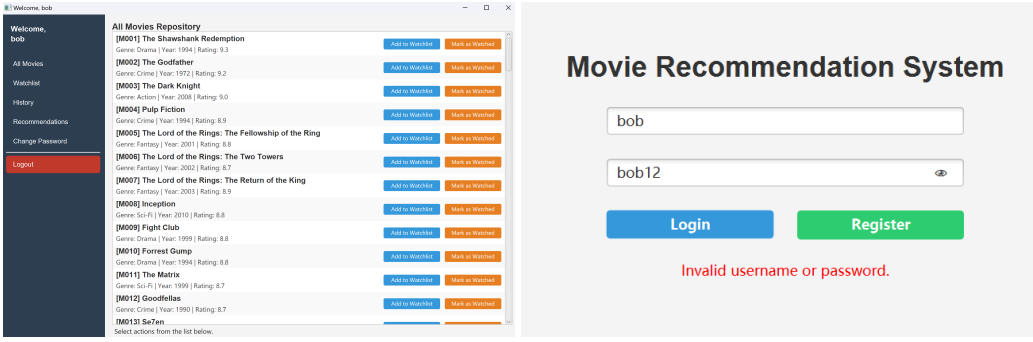


Figure 6: Login and registration workflows with input validation and error feedback

We tested the registration function ensure that identical usernames cannot be registered twice. If the two password entries fail to match during verification, registration will be unsuccessful. Upon successful registration, the user will be directed to the main page immediately.

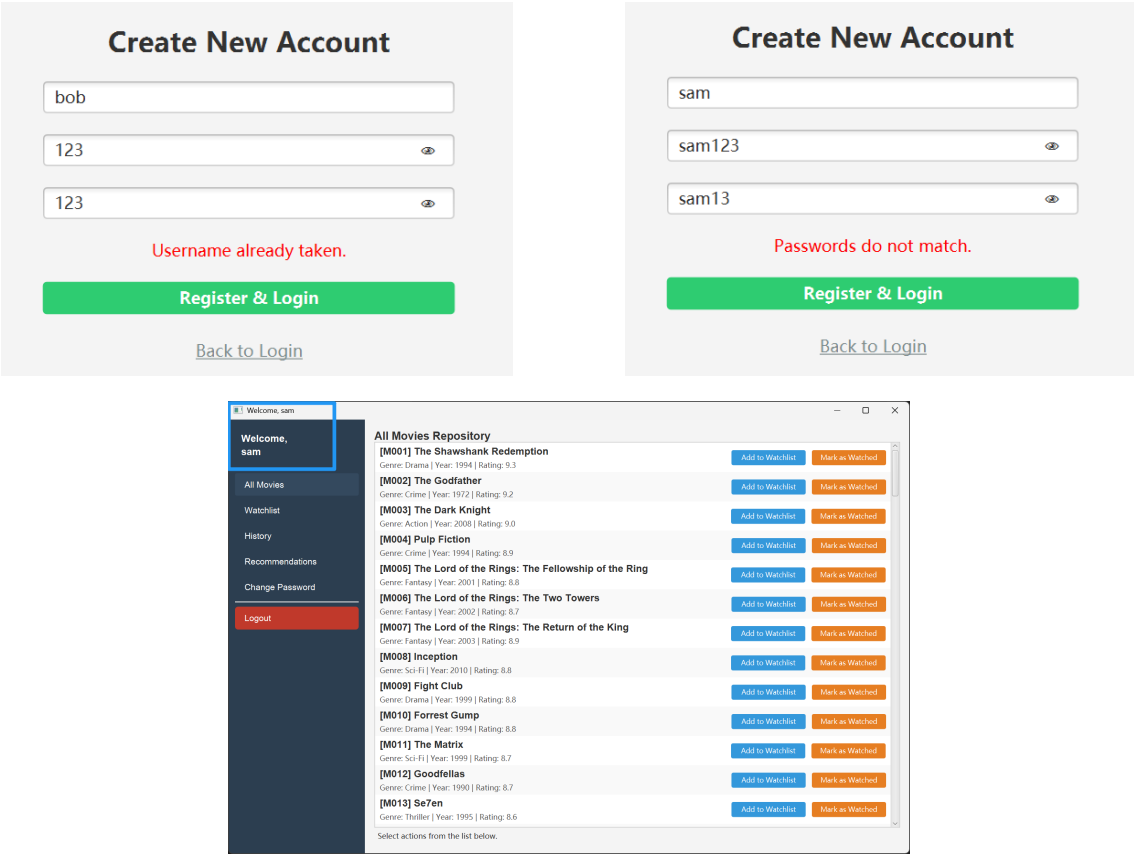


Figure 7: User authentication flow, including account creation validation and successful session entry

We also verified data persistence by adding a movie to the watchlist, closing the application, and checking whether the data has been saved to the CSV file. Additionally, state transitions were tested to ensure that clicking ”Mark as Watched” correctly moves a movie from the Watchlist to the History list. (use alice’s account to test, adding M001 to watchlist, marking M008 as watched, removing M015 from history)

Before

```
Username,Password,Watchlist,History
alice,h$cd97736fd86521c1,M008;M015;M071;M048;M056,M001@2025-07-12;M011@2025-08-10;M015@2025-09-01
```

After

```
Username,Password,Watchlist,History
alice,h$cd97736fd86521c1,M015;M071;M048;M056;M001,M001@2025-07-12;M011@2025-08-10;M008@2025-12-14
```

6.2 Algorithm Verification

The recommendation engine operates on a **closed feedback loop**. When a user adds a movie to the watchlist or marks one as watched, the system immediately updates the user profile, **recalculates all statistical metrics, regenerates the weights**, and produces a new recommendation list with updated scores. This mechanism ensures that the system **adapts in real-time** to the user's evolving interests. For example, we first generate 5 recommended movies based on currently watchlist and history, choosing a movie (M037) from the recommendation and adding it to watchlist. Then, if we try to generate another 5 recommended movies, the result will be different. The results are shown in the following two figures. Moreover, the recommendation engine will **not provide movies that already in your watchlist and history**.

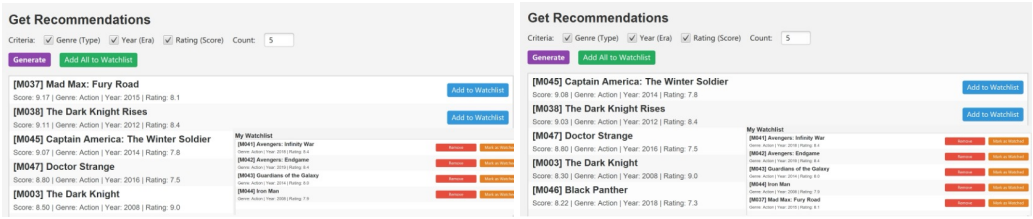


Figure 9: Algorithm Adaptability Test

7 Project Management & Ethical Assessment

7.1 Software Engineering Team Practices

Our team adopted an Agile development approach, allowing us to continuously refine the recommendation algorithms based on testing results and user feedback simulations. We utilized **Git** for version control with a **structured branching strategy** consisting of a main branch for production-ready code, a development branch for integration of completed features, and feature branches for individual component development. Code reviews were mandatory before merging, ensuring code quality and knowledge sharing across the team, which helped identify potential bugs early and maintained consistent coding standards throughout the project. Tasks were distributed based on team members' strengths and learning objectives, and we maintained comprehensive documentation including API documentation for all major components, database schema documentation, algorithm implementation details, and user manual for system operation. Regular documentation reviews ensured accuracy and completeness as the system evolved.

7.2 Ethical Assessment & Privacy Considerations

This section provides an ethical assessment of the Movie Recommendation System, focusing on potential risks related to data privacy, algorithmic bias, and the potential for misuse. The system's current design for data persistence presents significant privacy vulnerabilities, most critically in the way user passwords are handled. In the final implementation, passwords are **not stored in plaintext** within the `users.csv` file; instead, they are stored as one-way hashed values. During login, the system hashes the input password and compares it with the stored hash, which reduces the impact of data leakage because the original password cannot be directly recovered from the CSV file. Under coursework constraints (limited libraries and local CSV storage), this provides a practical improvement; in a production setting, a slow, industry-standard password hashing scheme with per-user random salts would be preferred. Furthermore, the collection of Watch History and Watchlist records, which store movie IDs in History and Watchlist respectively, can indirectly infer a user's personal beliefs or sensitive states. Therefore, when conducting large-scale data analysis or backup, these records must be anonymized from the user's direct identity, adhering to the principle of data minimization.

The recommendation engine raises an important ethical concern: algorithmic bias that can create filter bubbles and reduce diversity. In `RecommendationEngine.java`, the system calculates Genre Concentration and Year Variance to weight recommendation dimensions based on user preferences. While this personalizes results, it risks trapping users in a filter bubble where they only see movies similar to what they've already watched, limiting their exposure to new or niche content. To address this, the system should add serendipity or exploration features. This could work in two ways: automatically inject diverse movies or underrepresented genres into recommendations, or let users control a "Diversity/Exploration Slider" in the GUI to adjust how much the system prioritizes their historical preferences versus new discoveries.

Additionally, the system faces risks associated with its strategy for new users. The fallback mechanism for new users without history, defined in `getTopRatedMovies` within `RecommendationEngine.java`, relies on recommending a randomized subset of high-rated movies. If external entities could manipulate the base movie ratings, this strategy could be exploited to unfairly promote films to the most vulnerable user segment. Therefore, the source of the base movie rating data must be strictly audited to ensure impartiality, and the initial recommendations for new users should be diversified by balancing quality ratings with popularity metrics to ensure a broad and objective starting point.

Finally, the system needs to address Data Quality and Transparency. Currently, the recommendation results (the MovieScore) lack sufficient explanation, even though the `RecommendationEngine` does print the dynamic weights for analysis. Users only see a score without understanding why the recommendation was made, which erodes trust and hinders effective feedback. The solution is to enhance transparency by providing explanation blocks for each recommendation, detailing the calculated score and the specific contributing factors (e.g., "Recommended because your history is concentrated in Action type and close to your preferred year 2005"), and by visually presenting the weight contributions of Genre, Year, and Rating in the user interface.